

Detector de Matrícules

Universitat Politècnica de Catalunya
Facultat d'Informàtica de Barcelona

Visió per Computador

2025-2026 Q2

VC Grup: 11

Adrià Jara Palos: adria.jara@estudiantat.upc.edu

Eric Medina: eric.medina@estudiantat.upc.edu

Índex

1	Introducció	1
2	Descripció General del Funcionament	1
3	Implementació	3
3.1	Preprocessament i Generació de Candidats	3
3.1.1	Funcions Implementades	3
3.1.2	Decisions de Disseny	3
3.2	Classificador de Matrícules	3
3.2.1	Generació del Dataset	3
3.2.2	Funcions Implementades	4
3.2.3	Decisions de Disseny	4
3.2.4	Resultats i rendiment del classificador	5
3.2.4.1	Errors trobats	6
3.3	Neteja de Candidats Finals	6
3.3.1	Funcions Implementades	6
3.3.2	Decisions de Disseny	6
3.4	Alineació de la Matrícula Candidata	6
3.4.1	Funcions Implementades	6
3.4.2	Decisions de Disseny	6
3.5	Segmentació de Caràcters	6
3.5.1	Funcions Implementades	6
3.5.2	Decisions de Disseny	6
3.6	OCR	6
3.6.1	Dataset Utilitzat	6
3.6.2	Funcions Implementades	7
3.6.3	Decisions de Disseny*	7
3.6.4	Alternatives Descartades*	8
3.6.5	Resultats i rendiment del classificador	8
3.6.6	Errors trobats*	9
3.7	Resum del Flux i generació d'output	10
4	Resultats Finals	10
4.1	Exemples	10
5	Annexos	10
5.1	Codi Font	10
5.1.1	main	10
5.1.2	candidates_detection	10
5.1.3	plate_classifier	10
5.1.4	character_segmentation	10
5.1.5	dataset_generation	10
5.1.6	ocr_classifier	10

1. Introducció

El reconeixement automàtic de matrícules (*Automatic License Plate Recognition*, ALPR) és una aplicació clàssica dins l'àmbit de la visió per computador que permet identificar vehicles a partir de les imatges capturades per càmeres. Aquest tipus de sistemes tenen nombroses aplicacions pràctiques, com ara el control d'accessos, la gestió d'aparcaments, la monitorització del trànsit o els sistemes de vigilància viària.

L'objectiu d'aquesta pràctica és desenvolupar un sistema complet de detecció i reconeixement de matrícules utilitzant tècniques de visió per computador apreses a classe. El sistema implementat consta de diverses etapes consecutives: la detecció de possibles regions d'interès que poden contenir una matrícula, la classificació d'aquestes regions per descartar falsos positius, l'alineació geomètrica de la matrícula detectada, la segmentació dels caràcters individuals i, finalment, el reconeixement de cada caràcter.

2. Descripció General del Funcionament

El sistema desenvolupat segueix una arquitectura seqüencial formada per diverses etapes de processament, cadascuna de les quals té com a objectiu reduir progressivament la informació de la imatge fins a obtenir el text corresponent a la matrícula. La Figura 1 mostra de manera esquemàtica el flux general de processament implementat.

En primer lloc, a partir de la imatge original del vehicle es realitza una fase de detecció de regions candidates. L'objectiu d'aquesta etapa és localitzar totes les zones que podrien correspondre a una matrícula. Per fer-ho, s'utilitzen tècniques de processament d'imatge basades en la detecció de contorns i propietats geomètriques dels objectes presents a la imatge. Aquesta fase prioritza obtenir una elevada taxa de detecció, encara que això impliqui generar un cert nombre de falsos positius.

Les regions candidates obtingudes són posteriorment analitzades per un classificador binari (SVM) encarregat de determinar si una determinada regió conté una matrícula o no. Aquest classificador ha estat entrenat utilitzant descriptors visuals extrets de les imatges i permet descartar la major part de les deteccions incorrectes. Posteriorment, s'aplica una etapa de *Non-Maximum Suppression* (NMS) per eliminar deteccions redundants que corresponen a una mateixa matrícula.

Una vegada identificada la regió més probable, es procedeix a la seva alineació geomètrica. Aquesta etapa corregeix possibles inclinacions o deformacions produïdes per la perspectiva de la càmera, amb l'objectiu d'obtenir una imatge de la matrícula el més frontal possible.

A continuació es realitza la segmentació dels caràcters. En aquesta fase la imatge de la matrícula es converteix a escala de grisos i es binaritza. Posteriorment s'apliquen

diverses operacions morfològiques, com ara eliminació de soroll, tancament morfològic i filtratge per propietats geomètriques, amb la finalitat d'obtenir les regions corresponents als caràcters individuals. Els objectes detectats són filtrats segons criteris com la seva alçada relativa, relació d'aspecte i ocupació de la caixa contenidora. En cas de que la quantitat de caràcters detectats sigui inferior o igual a 3, es torna a realitzar l'alineació i segmentació amb la inversió de la imatge per intentar detectar matrícules amb fons negre i caràcters clars.

Una vegada segmentats els caràcters, es duu a terme el reconeixement de cadascun d'ells. Per representar cada símbol s'extreu un conjunt de característiques descriptives que inclouen descriptors HOG, perfils de projecció horitzontals i verticals, descriptors de densitat per zones, informació topològica i altres característiques estructurals. Aquest vector de característiques és introduït en un classificador basat en SVM, entrenat prèviament amb exemples de lletres i números.

Finalment, els caràcters reconeguts es concatenen seguint el seu ordre espacial dins de la matrícula per obtenir el text final detectat. Aquest resultat és comparat amb les anotacions disponibles al conjunt de dades per tal d'avaluar el rendiment global del sistema.

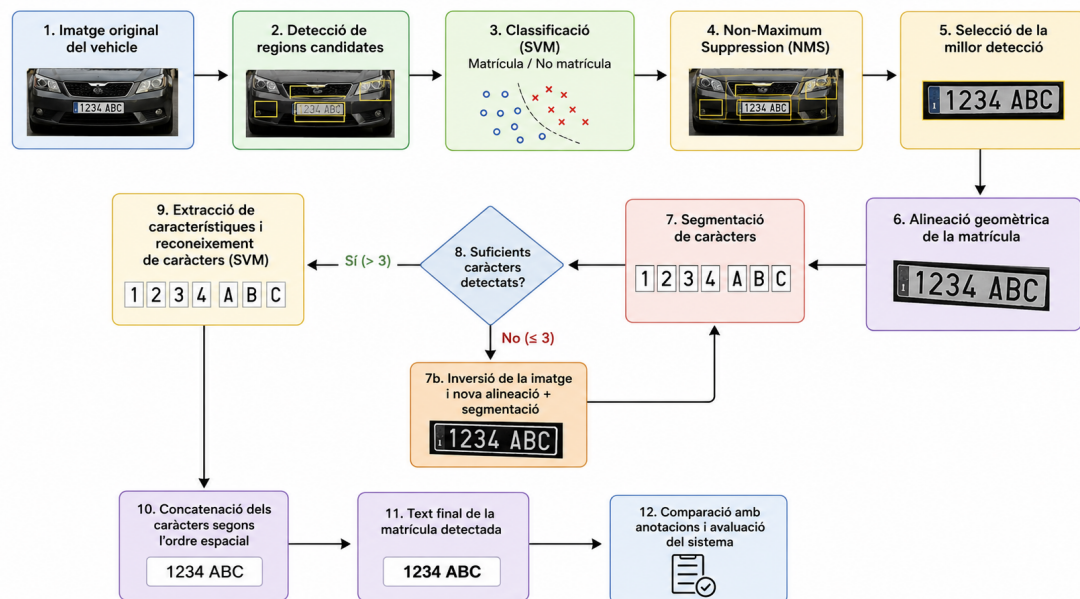


Figura 1: Esquema del pipeline de processament implementat per al reconeixement de matrícules.

3. Implementació

3.1. Preprocessament i Generació de Candidats

3.1.1. Funcions Implementades

3.1.2. Decisions de Disseny

3.2. Classificador de Matrícules

En aquesta etapa del projecte s'ha implementat un classificador binari encarregat de determinar si una regió d'imatge (ROI) conté o no una matrícula de vehicle. Cada imatge es transforma en un vector de característiques i posteriorment es classifica mitjançant un model (SVM) prèviament entrenat.

3.2.1. Generació del Dataset

Per entrenar el classificador s'ha utilitzat el dataset públic *Car License Plate Detection* disponible a Kaggle, el qual conté 433 imatges de vehicles amb les corresponents anotacions en format XML, que inclouen la posició de la matrícula dins de cada imatge.

Per tenir les dades en un format adequat per a l'entrenament del model, s'ha realitzat un procés de generació de mostres positives i negatives a partir de les imatges anotades mitjançant un script. L'script realitza les següents operacions per a cada imatge del dataset:

- Lectura de la imatge i del fitxer XML associat.
- Extracció de la bounding box de la matrícula.
- Aplicació d'un padding del 5 % per incloure context visual al voltant de la matrícula.
- Generació d'un exemple positiu mitjançant el retall directe de la matrícula.
- Generació de 5 exemples negatius mitjançant mostreig aleatori de regions de la imatge que no solapen amb la matrícula.

Aquests retalls són emmagatzemats en dues carpetes separades: una per a les mostres positives (matrícules) i una altra per a les mostres negatives (fons o objectes no rellevants).

Per garantir la qualitat dels exemples negatius, es calcula la mètrica Intersection over Union (IoU) entre la regió generada i la matrícula real. Només es conserven aquells crops amb IoU inferior a 0.1, evitant així mostres amb informació parcial de la matrícula.

3.2.2. Funcions Implementades

Les principals funcions implementades en aquest mòdul són:

- **compute_iou**: funció exclusiva de la part de generació del dataset. Donades dues bounding boxes, calcula la intersecció sobre la unió (IoU) d'aquestes per assegurar que els exemples negatius no solapen amb la matrícula. D'aquesta forma es garanteix que el classificador aprengui a diferenciar entre matrícules i fons.
- **extract_plate_features**: funció compartida entre la part d'entrenament i la part de classificació. Extreu el vector de característiques de cada imatge (explicades al següent apartat).
- **classify_roi**: funció encarregada de carregar el model SVM entrenat i extreure les característiques i classificar una regió d'interès (ROI) donada. Aquesta funció s'utilitza durant la fase de detecció per filtrar les regions candidates obtingudes a partir de les tècniques de processament d'imatge del mòdul anterior.

A part d'aquestes funcions principals, s'han implementat els scripts que gestionen tot el procés:

- **generate_dataset**: script encarregat de processar les imatges anotades i generar els crops positius i negatius (com explicat en l'apartat anterior).
- **train_plate_classifier**: script que extreu les característiques de cada imatge amb la funció **extract_plate_features**, divideix els conjunts de features y labels en conjunts de entrenament (80 %) i test (20 %), i entrena un model SVM amb kernel lineal. Finalment calcula i mostra les mètriques d'avaluació (accuracy, precision, recall i matriu de confusió) sobre el conjunt de test.

Totes aquestes funcions i scripts es poden trobar a l'annex.

3.2.3. Decisions de Disseny

Primer de tot, en les decisions de disseny del dataset d'entrenament, s'ha optat per buscar un d'internet ja que el dataset donat per l'entrega era massa petit (només 100 imatges) per entrenar un model de classificació robust. A més, es va decidir generar exemples negatius a partir de les mateixes imatges del dataset per garantir que el model aprengué a diferenciar entre matrícules i fons en el mateix context visual (portes, carrers, llums, etc). També es va decidir aplicar un padding del 5 % a les bounding boxes de les matrícules per incloure una mica de context visual al voltant d'aquestes.

En el disseny del classificador s'ha optat per un enfocament basat en *Support Vector Machines* (SVM) amb kernel lineal, degut a la seva bona capacitat de generalització en problemes de classificació binària amb vectors de característiques d'alta dimensió.

Pel que fa a la representació de les imatges, s'ha escollit els següents descriptors:

- **HOG (Histogram of Oriented Gradients):** com les matrícules tenen una estructura visual molt característica (línies horitzontals i verticals, contrast entre caràcters i fons), els descriptors HOG són molt adequats per capturar aquesta informació. S'han utilitzat 9 orientacions i cel·les de 8x8 píxels.
- **Aspect ratio:** relació entre amplada i alçada de la regió, que ajuda a diferenciar matrícules de regions quadrades o irregulars.
- **Area total:** mida de la regió, que aporta informació addicional sobre l'escala de l'objecte.

Inicialment només es van utilitzar els descriptors HOG. Tot i tenir una bona performance inicial, després de les proves es va observar que el model, en moments puntuals detectava regions amb aspect ratio molt diferent al d'una matrícula (per exemple, regions molt quadrades o molt allargades). Per aquest motiu es van afegir les característiques d'aspect ratio i area total. No van ser de gran ajuda degut a la dimensionalitat de Hough respecte a aquestes dues característiques, però es van decidir mantenir.

Totes les imatges es redimensionen a una mida fixa abans de l'extracció de característiques per garantir consistència en els descriptors. Es va decidir utilitzar una mida de 128x64 píxels.

3.2.4. Resultats i rendiment del classificador

Per avaluar el rendiment del classificador SVM s'ha utilitzat un conjunt de test independent del conjunt d'entrenament (del 20 %) i s'ha establert una seed fixa (42) per reproduir els resultats. La mètrica principal utilitzada ha sigut la matriu de confusió, d'on s'han tret també les mètriques de accuracy, precision i recall.

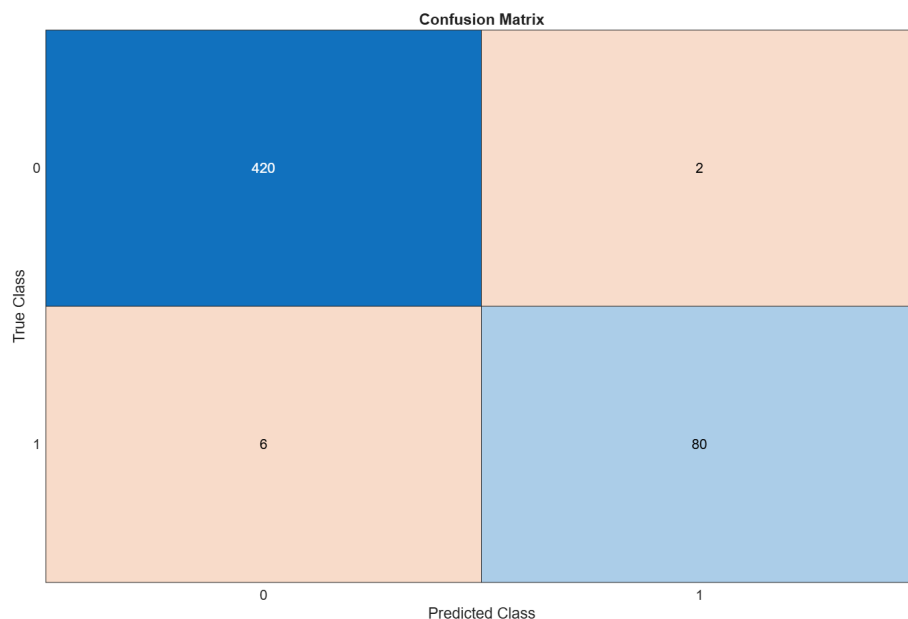


Figura 2: Matriu de confusió del classificador SVM

Després es van calcular les següents mètriques:

- **Accuracy:** 98.62 %
- **Precision:** 97.56 %
- **Recall:** 94.12 %

Els resultats obtinguts mostren una capacitat de discriminació adequada entre les classes positives (matrícules) i negatives (no matrícules).

3.2.4.1 Errors trobats

Els principals errors es produeixen en regions amb característiques visuals similars a les matrícules, com ara parts del vehicle amb formes rectangulars (reixes, filtres, etc.) o text en el fons.

Tot i això, posteriorment es fa un tractament d'aquests casos a través que s'explica a continuació.

3.3. Neteja de Candidats Finals

3.3.1. Funcions Implementades

3.3.2. Decisions de Disseny

3.4. Alineació de la Matrícula Candidata

3.4.1. Funcions Implementades

3.4.2. Decisions de Disseny

3.5. Segmentació de Caràcters

3.5.1. Funcions Implementades

3.5.2. Decisions de Disseny

3.6. OCR

En aquesta etapa del sistema s'ha implementat un classificador OCR (SVM) encarregat d'identificar cada caràcter individual segmentat de la matrícula (lletres i números). A partir dels blobs obtinguts en la fase de segmentació, el sistema assigna una etiqueta a cada regió mitjançant un model entrenat prèviament.

3.6.1. Dataset Utilitzat

Per a l'entrenament del classificador OCR s'ha utilitzat un dataset públic d'OCR disponible a Kaggle (**standard OCR dataset**), el qual conté imatges de caràcters individuals (lletres i números) organitzades en carpetes separades per classe. Cada

classe conté aproximadament entre 500 i 600 imatges, proporcionant una distribució equilibrada entre caràcters.

El dataset s'ha carregat utilitzant un `imageDatastore` de MATLAB, configurat per llegir automàticament les subcarpetes i assignar les etiquetes segons el nom de cada carpeta.

3.6.2. Funcions Implementades

Les principals funcions implementades en aquest mòdul són:

- **`extract_character_features`**: funció compartida entre l'entrenament i la part de classificació. S'encarregada d'extreure el vector de característiques de cada caràcter (explicades al següent apartat).
- **`classify_character`**: funció que carrega el model SVM entrenat, extreu les característiques i retorna la predicció del caràcter corresponent a una imatge d'entrada.
- **`recognize_plate`**: funció que recorre tots els blobs detectats en una matrícula segmentada, aplica el classificador (`classify_character`) a cada regió i construeix la cadena final de la matrícula concatenant les prediccions.

A part d'aquestes funcions principals, s'ha implementat el script que gestiona l'entrenament del model:

- **`train_ocr_classifier`**: script que carrega el dataset d'OCR, divideix els conjunts de features y labels en conjunts de entrenament (80 %) i test (20 %), extreu les característiques de cada imatge amb la funció **`extract_character_features`** i entrena un model SVM amb kernel lineal. Finalment calcula i mostra les mètriques d'avaluació (accuracy i matriu de confusió) sobre el conjunt de test.

3.6.3. Decisions de Disseny*

Primer de tot, respecte la selecció del dataset d'entrenament, s'han utilitzat diverses alternatives al llarg del procés. Inicialment es va utilitzar un altre dataset (també d'internet) el qual contenia imatges de caràcters directament obtingut de matrícules reals. Tot i això, es van observar diversos problemes després de fer les primeres proves. Les causes indentificades van ser que totes les imatges (de cada caràcter) eren de la mateixa tipologia, per tant, el model no era capaç de generalitzar a altres fonts d'imatges de forma correcta. A més, moltes estaven rotades o deformades, i en el nostre cas això perjudicava més que ajudava ja que en el nostre pipeline es realitza una etapa d'alineació prèvia a la segmentació, per tant, els caràcters sempre apareixien rectes (sense rotacions exagerades). Després de descartar aquest dataset, es va optar per utilitzar el dataset d'OCR clàssic mencionat anteriorment.

Per a la representació dels caràcters s'han combinat múltiples descriptors amb l'objectiu de capturar tant la forma global com les estructures internes dels símbols:

-
- **HOG (Histogram of Oriented Gradients)**: utilitzat amb cel·les de 4x4 píxels per capturar millor els detalls fins dels caràcters, com traços i cantonades. Inicialment es van provar cel·les de 8x8, però va reduir a 4x4 degut a la mida reduïda de les imatges.
 - **Densitat global**: proporció de píxels actius dins la imatge binaritzada, que ajuda a diferenciar caràcters plens de caràcters oberts.
 - **Nombre de components connectats**: mesura del nombre de components connectats de la imatge binaritzada negada, que pot ajudar a distingir caràcters amb forats (per exemple B i 8).
 - **Perfils de projecció**: distribució horitzontal i vertical dels píxels, normalitzada a longitud fixa, que permet capturar l'estructura general del caràcter.
 - **Zoning 4x4 i 6x3**: divisió de la imatge en regions per calcular densitat local, aportant informació espacial sobre la distribució dels traços.
 - **Crossing numbers**: nombre de transicions entre fons i primer pla per fila, útil per distingir caràcters amb estructures similars.
 - etc. Per determinar

Totes les imatges es redimensionen a una mida fixa de 32x32 píxels abans de l'extracció de característiques per garantir consistència entre mostres.

3.6.4. Alternatives Descartades*

3.6.5. Resultats i rendiment del classificador

El rendiment del model s'ha avaluat utilitzant un conjunt de test independent (20 % del dataset total), amb una divisió aleatòria de les dades. La mètrica principal utilitzada ha sigut la matriu de confusió, d'on s'han tret també la mètrica de accuracy.

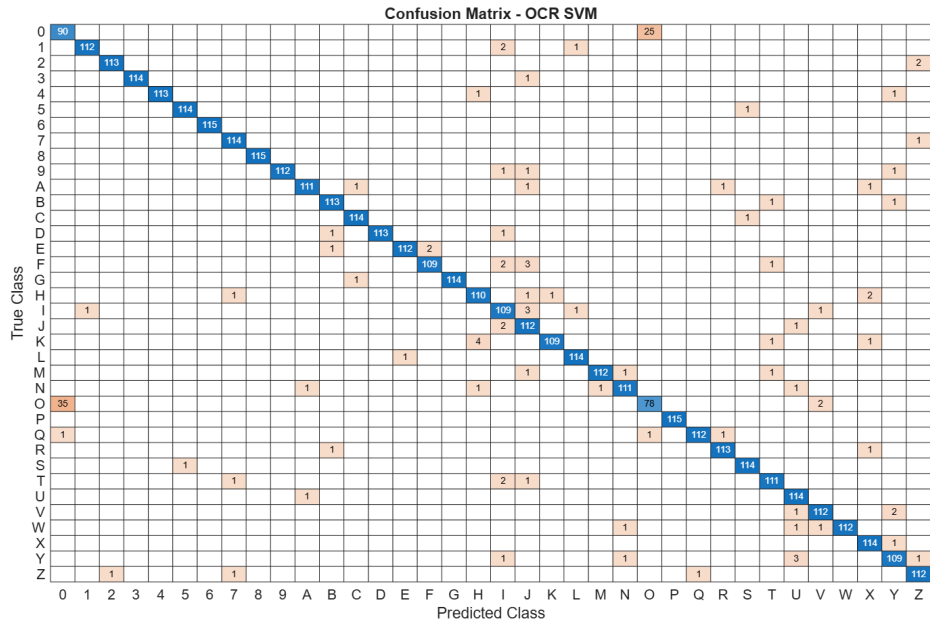


Figura 3: Matriu de confusió del classificador OCR

- Accuracy: 96.72 %

Com es pot observar a la matriu de confusió, el model té una bona capacitat de discriminació entre les diferents classes de caràcters, tot i que hi han casos concrets on li costa diferenciar depèn de quin caràcter. On més es pot identificar aquest problema és entre el 0 i la O, amb una quantitat considerable de confusions. Els demès errors es distribueixen de forma més dispersa i puntual.

3.6.6. Errors trobats*

Tot i haver obtingut un bon rendiment amb el model de test, en les proves reals realitzades (conjuntament amb tot el pipeline) s'han detectat diversos errors de reconeixement.

Les lletres més conflictives han sigut:

- **0 vs O:** com s'ha mencionat anteriorment, el model té dificultats per diferenciar entre el número 0 i la lletra O, especialment quan les matrícules tenen una tipografia on aquests símbols són molt similars.
- **W vs H:**

3.7. Resum del Flux i generació d'output

4. Resultats Finals

4.1. Exemples

5. Annexos

5.1. Codi Font

5.1.1. main

5.1.2. candidates_detection

5.1.3. plate_classifier

5.1.4. character_segmentation

5.1.5. dataset_generation

5.1.6. ocr_classifier